

Министерство образования и науки Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”

Факультет программной инженерии и компьютерной техники (ПИКТ)

Направление подготовки (специальность) – 09.03.04 (Нейротехнологии и
программирование)

Информатика

Лабораторная работа № 4

“Введение в базы данных”

Выполнил

студент

Попов Глеб Ильич

Группа № Р3121

Преподаватель: Болдырева Елена Александровна

г. Санкт-Петербург

2024 г.

Оглавление

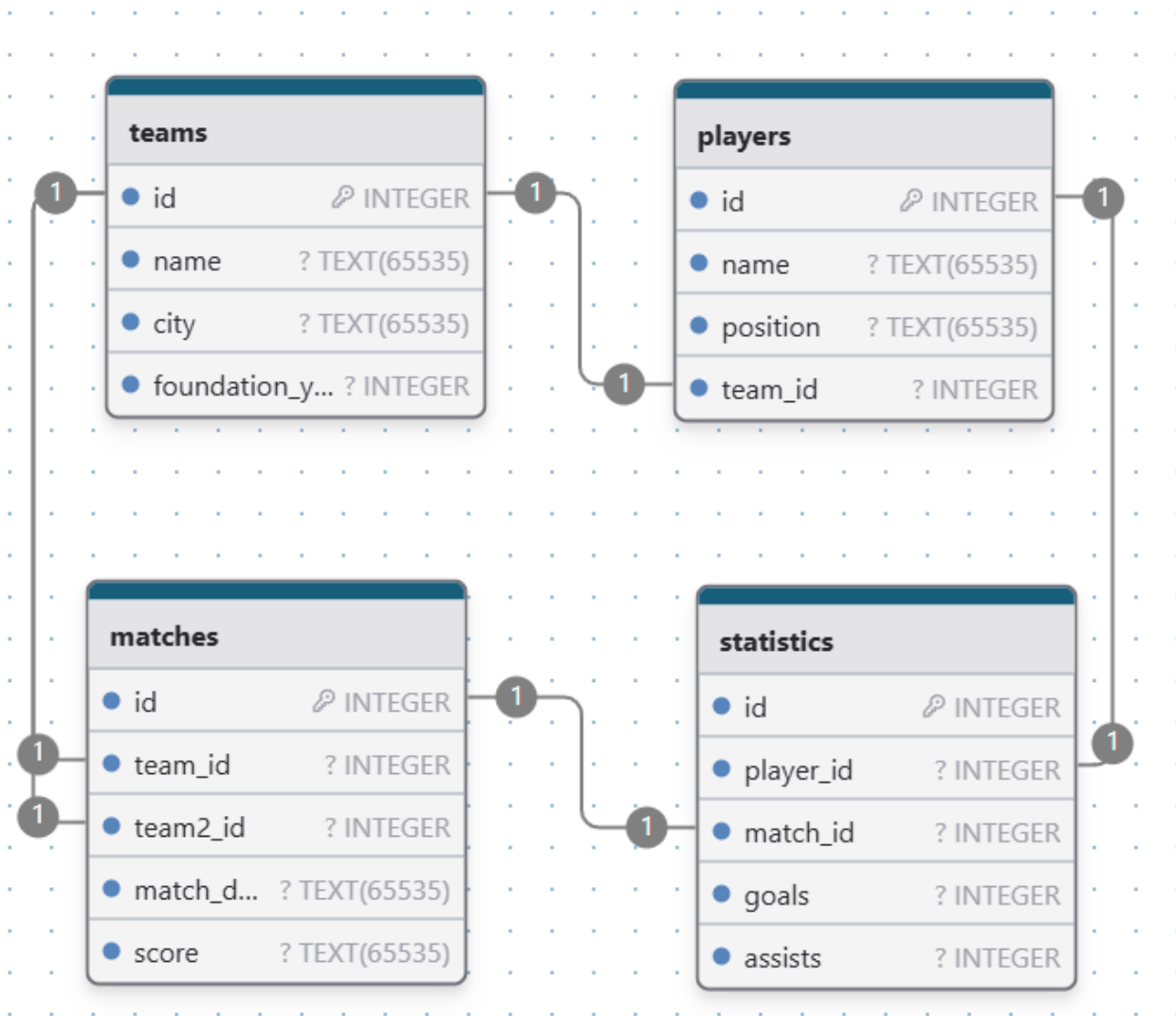
Задание:	3
Основные этапы вычисления:	3
Вывод:	17
Список литературы:	17

Задание:

- 1) Прodelайте все шаги из примера. Удостоверьтесь, что Вы поняли, как это работает.
- 2) Самостоятельно выберите тематику для своей собственной базы данных. База данных должна содержать не менее четырех таблиц. Каждая таблица должна иметь минимум одну связь с другими таблицами для формирования сложных запросов. Каждая таблица содержит не менее 5 записей.
- 3) Нарисуйте схему базы данных как в примере с указанием связей между таблицами.
- 4) Создайте таблицы с помощью Python для SQLite и MySQL.
- 5) Добавьте по одной новой записи в каждую из таблиц Вашей базы данных.
- 6) Продемонстрируйте работу запросов на извлечение данных:
 - выбрать все записи из таблиц
 - составить запрос по извлечению данных с использованием JOIN
 - составить запрос по извлечению данных с использованием WHERE и GROUP BY
 - составить два запроса, в которых будет вложенный SELECT-запрос (вложение с помощью WHERE
 - составить 2 запроса с использованием UNION (объединение запросов)
 - составить 1 запрос с использованием DISTINCT. Если для демонстрации работы этого ключевого слова недостаточно данных – предварительно дополните таблицу
- 7) Обновить две записи в двух разных таблицах Вашей базы данных.
- 8) Удалить по одной записи из каждой таблицы.
- 9) Удалите все записи в одной из таблиц.

Основные этапы вычисления:

- 1) Прodelал, убедился.
- 2) Выбрал, тематика – хоккейные команды



SQLite:

```

import sqlite3
from sqlite3 import Error

# Подключение к базе данных SQLite
1 usage
def create_connection(db_file):
    connection = None
    try:
        connection = sqlite3.connect(db_file)
        print("Connection to SQLite DB successful")
    except Error as e:
        print(f"The error '{e}' occurred")
    return connection

# Выполнение SQL-запроса
12 usages
def execute_query(connection, query):
    if connection is None:
        raise Exception("Connection to the database was not established.")
    cursor = connection.cursor()
    try:
        cursor.execute(query)
        connection.commit()
        print("Query executed successfully")
    except Error as e:
        print(f"The error '{e}' occurred")

# Чтение данных из базы
6 usages
def execute_read_query(connection, query):
    if connection is None:
        raise Exception("Connection to the database was not established.")
    cursor = connection.cursor()
    result = None
    try:
        cursor.execute(query)
        result = cursor.fetchall()
        return result
    except Error as e:
        print(f"The error '{e}' occurred")

```

```

# Подключение к базе данных SQLite
connection = create_connection("hockey_app.sqlite")
if connection is None:
    raise Exception("Failed to connect to the SQLite database.")

# Создание таблиц
create_teams_table = """
CREATE TABLE IF NOT EXISTS teams (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    city TEXT NOT NULL,
    foundation_year INTEGER
);
"""

create_players_table = """
CREATE TABLE IF NOT EXISTS players (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    position TEXT NOT NULL,
    team_id INTEGER NOT NULL,
    FOREIGN KEY (team_id) REFERENCES teams (id)
);
"""

create_matches_table = """
CREATE TABLE IF NOT EXISTS matches (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    team1_id INTEGER NOT NULL,
    team2_id INTEGER NOT NULL,
    match_date TEXT,
    score TEXT,
    FOREIGN KEY (team1_id) REFERENCES teams (id),
    FOREIGN KEY (team2_id) REFERENCES teams (id)
);
"""

```

```

create_statistics_table = """
CREATE TABLE IF NOT EXISTS statistics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    player_id INTEGER NOT NULL,
    match_id INTEGER NOT NULL,
    goals INTEGER,
    assists INTEGER,
    FOREIGN KEY (player_id) REFERENCES players (id),
    FOREIGN KEY (match_id) REFERENCES matches (id)
);
"""

execute_query(connection, create_teams_table)
execute_query(connection, create_players_table)
execute_query(connection, create_matches_table)
execute_query(connection, create_statistics_table)

# Добавление данных
insert_teams = """
INSERT INTO
    teams (name, city, foundation_year)
VALUES
    ('Ice Titans', 'New York', 1990),
    ('Frost Giants', 'Los Angeles', 1985),
    ('Blizzard Hawks', 'Chicago', 2000),
    ('Polar Bears', 'Houston', 1995),
    ('Arctic Wolves', 'Miami', 1980);
"""

insert_players = """
INSERT INTO
    players (name, position, team_id)
VALUES
    ('Alex Johnson', 'Center', 1),
    ('Bobby Smith', 'Left Wing', 2),
    ('Charlie Brown', 'Right Wing', 3),
    ('David White', 'Defenseman', 4),
    ('Ethan Black', 'Goalie', 5);
"""

```

```

insert_matches = """
INSERT INTO
    matches (team1_id, team2_id, match_date, score)
VALUES
    (1, 2, '2024-03-01', '4-3'),
    (3, 4, '2024-03-05', '2-2'),
    (5, 1, '2024-03-10', '1-3'),
    (2, 3, '2024-03-15', '3-4'),
    (4, 5, '2024-03-20', '2-1');
"""

insert_statistics = """
INSERT INTO
    statistics (player_id, match_id, goals, assists)
VALUES
    (1, 1, 2, 1),
    (2, 1, 1, 0),
    (3, 2, 0, 2),
    (4, 2, 0, 1),
    (5, 3, 1, 0);
"""

execute_query(connection, insert_teams)
execute_query(connection, insert_players)
execute_query(connection, insert_matches)
execute_query(connection, insert_statistics)

# Запросы для извлечения данных
# Выбрать все записи из таблиц
select_teams = "SELECT * FROM teams;"
teams = execute_read_query(connection, select_teams)
print("Teams:", teams)

select_players = "SELECT * FROM players;"
players = execute_read_query(connection, select_players)
print("Players:", players)

```



```

# Запрос с JOIN
select_match_results = """
SELECT
    matches.id,
    t1.name AS team1,
    t2.name AS team2,
    matches.match_date,
    matches.score
FROM
    matches
    INNER JOIN teams t1 ON matches.team1_id = t1.id
    INNER JOIN teams t2 ON matches.team2_id = t2.id;
"""

match_results = execute_read_query(connection, select_match_results)
print("Match Results:", match_results)

# Запрос с WHERE и GROUP BY
select_player_statistics = """
SELECT
    players.name,
    SUM(statistics.goals) AS total_goals,
    SUM(statistics.assists) AS total_assists
FROM
    statistics
    INNER JOIN players ON statistics.player_id = players.id
GROUP BY
    players.name;
"""

player_statistics = execute_read_query(connection, select_player_statistics)
print("Player Statistics:", player_statistics)

# UNION запросы
union_query = """
SELECT name FROM teams WHERE city = 'New York'
UNION
SELECT name FROM teams WHERE foundation_year < 1990;

```

```

union_result = execute_read_query(connection, union_query)
print("Union Result:", union_result)

# DISTINCT запрос
distinct_positions = "SELECT DISTINCT position FROM players;"
positions = execute_read_query(connection, distinct_positions)
print("Distinct Positions:", positions)

# Обновление данных
update_player_position = """
UPDATE
    players
SET
    position = 'Center Forward'
WHERE
    id = 1;
"""
execute_query(connection, update_player_position)

update_team_city = """
UPDATE
    teams
SET
    city = 'San Francisco'
WHERE
    id = 2;
"""
execute_query(connection, update_team_city)

# Удаление данных
delete_statistics = "DELETE FROM statistics WHERE id = 5;"
execute_query(connection, delete_statistics)

delete_all_matches = "DELETE FROM matches;"
execute_query(connection, delete_all_matches)

```

MySQL:

```

import mysql.connector
from mysql.connector import Error

# Подключение к базе данных

def create_connection(host_name, user_name, user_password):
    connection = None
    try:
        connection = mysql.connector.connect(
            host=host_name,
            user=user_name,
            password=user_password
        )
        if connection.is_connected():
            print("Connection to MySQL server successful")
    except Error as e:
        print(f"The error '{e}' occurred")
    return connection

# Подключение к конкретной базе данных

def create_db_connection(host_name, user_name, user_password, db_name):
    connection = None
    try:
        connection = mysql.connector.connect(
            host=host_name,
            user=user_name,
            password=user_password,
            database=db_name
        )
        if connection.is_connected():
            print("Connection to MySQL DB successful")
    except Error as e:
        print(f"The error '{e}' occurred")
    return connection

# Выполнение SQL-запроса
def execute_query(connection, query):
    if connection is None:
        raise Exception("Connection to the database was not established.")
    cursor = connection.cursor()
    try:
        cursor.execute(query)
        connection.commit()
        print("Query executed successfully")
    except Error as e:
        print(f"The error '{e}' occurred")

```

```

# Чтение данных из базы
def execute_read_query(connection, query):
    if connection is None:
        raise Exception("Connection to the database was not established.")
    cursor = connection.cursor()
    result = None
    try:
        cursor.execute(query)
        result = cursor.fetchall()
        return result
    except Error as e:
        print(f"The error '{e}' occurred")

# Подключение к серверу MySQL
connection = create_connection("localhost", "root", "password")
if connection is None:
    raise Exception("Failed to connect to the MySQL server. Check your credentials or server status.")

# Создание базы данных
create_database_query = "CREATE DATABASE IF NOT EXISTS hockey_app;"
execute_query(connection, create_database_query)

# Подключение к базе данных
connection = create_db_connection("localhost", "root", "password", "hockey_app")
if connection is None:
    raise Exception("Failed to connect to the 'hockey_app' database. Ensure the database exists and credentials are correct.")

# Создание таблиц
create_teams_table = """
CREATE TABLE IF NOT EXISTS teams (
    id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    foundation_year INT,
    PRIMARY KEY (id)
);
"""

create_players_table = """
CREATE TABLE IF NOT EXISTS players (
    id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    position VARCHAR(100) NOT NULL,
    team_id INT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (team_id) REFERENCES teams (id)
);
"""

```

```

create_matches_table = """
CREATE TABLE IF NOT EXISTS matches (
    id INT AUTO_INCREMENT,
    team1_id INT NOT NULL,
    team2_id INT NOT NULL,
    match_date DATE,
    score VARCHAR(20),
    PRIMARY KEY (id),
    FOREIGN KEY (team1_id) REFERENCES teams (id),
    FOREIGN KEY (team2_id) REFERENCES teams (id)
);
"""

create_statistics_table = """
CREATE TABLE IF NOT EXISTS statistics (
    id INT AUTO_INCREMENT,
    player_id INT NOT NULL,
    match_id INT NOT NULL,
    goals INT,
    assists INT,
    PRIMARY KEY (id),
    FOREIGN KEY (player_id) REFERENCES players (id),
    FOREIGN KEY (match_id) REFERENCES matches (id)
);
"""

execute_query(connection, create_teams_table)
execute_query(connection, create_players_table)
execute_query(connection, create_matches_table)
execute_query(connection, create_statistics_table)

# Добавление данных
insert_teams = """
INSERT INTO
    teams (name, city, foundation_year)
VALUES
    ('Ice Titans', 'New York', 1990),
    ('Frost Giants', 'Los Angeles', 1985),
    ('Blizzard Hawks', 'Chicago', 2000),
    ('Polar Bears', 'Houston', 1995),
    ('Arctic Wolves', 'Miami', 1980);
"""

```

```

insert_players = """
INSERT INTO
|  players (name, position, team_id)
VALUES
|  ('Alex Johnson', 'Center', 1),
|  ('Bobby Smith', 'Left Wing', 2),
|  ('Charlie Brown', 'Right Wing', 3),
|  ('David White', 'Defenseman', 4),
|  ('Ethan Black', 'Goalie', 5);
"""

insert_matches = """
INSERT INTO
|  matches (team1_id, team2_id, match_date, score)
VALUES
|  (1, 2, '2024-03-01', '4-3'),
|  (3, 4, '2024-03-05', '2-2'),
|  (5, 1, '2024-03-10', '1-3'),
|  (2, 3, '2024-03-15', '3-4'),
|  (4, 5, '2024-03-20', '2-1');
"""

insert_statistics = """
INSERT INTO
|  statistics (player_id, match_id, goals, assists)
VALUES
|  (1, 1, 2, 1),
|  (2, 1, 1, 0),
|  (3, 2, 0, 2),
|  (4, 2, 0, 1),
|  (5, 3, 1, 0);
"""

execute_query(connection, insert_teams)
execute_query(connection, insert_players)
execute_query(connection, insert_matches)
execute_query(connection, insert_statistics)

# Запросы для извлечения данных
# Выбрать все записи из таблиц
select_teams = "SELECT * FROM teams;"
teams = execute_read_query(connection, select_teams)
print("Teams:", teams)

```

```

select_players = "SELECT * FROM players;"
players = execute_read_query(connection, select_players)
print("Players:", players)

# Запрос с JOIN
select_match_results = """
SELECT
    matches.id,
    t1.name AS team1,
    t2.name AS team2,
    matches.match_date,
    matches.score
FROM
    matches
    INNER JOIN teams t1 ON matches.team1_id = t1.id
    INNER JOIN teams t2 ON matches.team2_id = t2.id;
"""

match_results = execute_read_query(connection, select_match_results)
print("Match Results:", match_results)

# Запрос с WHERE и GROUP BY
select_player_statistics = """
SELECT
    players.name,
    SUM(statistics.goals) AS total_goals,
    SUM(statistics.assists) AS total_assists
FROM
    statistics
    INNER JOIN players ON statistics.player_id = players.id
GROUP BY
    players.name;
"""

player_statistics = execute_read_query(connection, select_player_statistics)
print("Player Statistics:", player_statistics)

# UNION запросы
union_query = """
SELECT name FROM teams WHERE city = 'New York'
UNION
SELECT name FROM teams WHERE foundation_year < 1990;
"""

```

```

union_result = execute_read_query(connection, union_query)
print("Union Result:", union_result)

# DISTINCT запрос
distinct_positions = "SELECT DISTINCT position FROM players;"
positions = execute_read_query(connection, distinct_positions)
print("Distinct Positions:", positions)

# Обновление данных
update_player_position = """
UPDATE
|   players
SET
|   position = 'Center Forward'
WHERE
|   id = 1;
"""

execute_query(connection, update_player_position)

update_team_city = """
UPDATE
|   teams
SET
|   city = 'San Francisco'
WHERE
|   id = 2;
"""

execute_query(connection, update_team_city)

# Удаление данных
delete_statistics = "DELETE FROM statistics WHERE id = 5;"
execute_query(connection, delete_statistics)

delete_all_matches = "DELETE FROM matches;"
execute_query(connection, delete_all_matches)

```


Вывод:

В результате данной лабораторной работы я узнал, как подключаться к СУБД с помощью библиотек Python для работы с SQL базами данных, как управлять базами данных SQLite, как выполнять запросы к базе данных внутри приложения Python.

Список литературы:

1. Балакшин П.В., Соснин В.В., Машина Е.А. Информатика.– СПб: Университет ИТМО, 2020.– 122 с.
2. Орлов С. А., Цилькер Б. Я. Организация ЭВМ и систем: Учебник для вузов. 2-е изд. – СПб.: Питер, 2011. – 688 с.: ил.
3. Алексеев Е.Г., Богатырев С.Д. Информатика. Мультимедийный электронный учебник.